

AULA 11

O Pong em classes

Entidade, herança e a classe World — o mesmo jogo, outra organização

HOJE

Um código só, dissecado

1. O Pong reescrito em classes — pronto e rodando.
2. Onde cada coisa foi morar, parte por parte.
3. Onde a herança já começa a incomodar.

Ninguém digita hoje. Refatorar o próprio Pong é a tarefa de casa.

A linha que queríamos escrever

```
for entidade in entidades:  
    entidade.update(PASSO)
```

Ela existe. Hoje ela roda.

Para isso: cada coisa carrega o seu estado, e todas respondem ao mesmo chamado.

ANTES DE PEDIR

O que muda e o que não pode mudar

Muda

a organização: classes, herança

Não muda

o que aparece na tela, quadro a quadro

as velocidades, em pixels por segundo

o laço de passo fixo da aula passada

os defeitos que o código já tinha

Especificar antes de pedir. De novo.

O prompt

Reescreva este Pong em classes.

Entidade (abstrata): update(dt) e desenhar(tela).

Bola, Raquete -> RaqueteDoJogador e RaqueteDoOponente.

World guarda as entidades, o placar e o laço.

Colisão e ponto ficam em World.fisica(), depois do for.

Não mude o comportamento na tela. Nem as velocidades.

Modalidade âmbar: o código é da IA, a crítica é nossa.

A tela é a mesma

Mesma bola, mesmas raquetes, mesmo placar — passo a passo. Vinte mil passos comparados entre o código de sexta e o de hoje: idênticos.

Se a tela mudou, não foi refatoração. Foi outra coisa.

Onde foi parar o while

```
class World:
    def game_loop(self):
        acumulador = 0
        while self.rodando:
            acumulador += min(self.clock.tick(240) / 1000, 0.25)
            self.inputs()
            while acumulador >= PASSO:
                self.update(PASSO)
                acumulador -= PASSO
            self.desenhar()
```

O laço de sexta, inteiro, sem uma linha a menos. Só mudou de endereço.

O que sumiu de dentro dele

doze variáveis globais → nenhuma

um update() que sabia de tudo → World.update(), que só chama

inputs(), update() e draw() soltos → métodos de World

Nada do jogo mora mais fora de uma classe.

__init__ e self

```
class Bola(Entidade):  
    def __init__(self):  
        self.x = 400  
        self.y = 300  
        self.vx = 300  
        self.vy = 0  
  
    def update(self, dt):  
        self.x += self.vx * dt  
        self.y += self.vy * dt
```

bola_x virou self.x. A diferença é que agora existe uma bola, não quatro variáveis.

Raquete e as duas filhas

```
class Raquete(Entidade):  
    def limitar_na_tela(self): ...  
    def desenhar(self, tela): ...  
  
class RaqueteDoJogador(Raquete):  
    def update(self, dt): # lê o teclado  
  
class RaqueteDoOponente(Raquete):  
    def update(self, dt): # segue a bola
```

O que é comum fica em um lugar só. Muda quem decide o movimento.

Dois motivos diferentes para herdar

Entidade → Bola, Raquete

contrato: todo mundo na lista responde a update()

Raquete → RaqueteDoJogador, RaqueteDoOponente

reaproveitamento: desenho, dimensões e trava de tela

O primeiro andar o Python não exige. O segundo se paga em qualquer linguagem.

O contrato morde

```
>>> Raquete(30)  
TypeError: Can't instantiate abstract class Raquete  
with abstract method update
```

“Não existe sozinha” deixa de ser figura de linguagem.

É a resposta da pergunta de sexta: em Python, a classe abstrata serve para isto.

PERGUNTA

De quem é a colisão?

A bola bate na raquete. Esse código mora em qual classe?

RESPOSTA

De nenhuma entidade. É de uma etapa.

```
class World:  
  
    def update(self, dt):  
        for entidade in self.entidades:  
            entidade.update(dt)  
  
        self.fisica()
```

Colisão é relação entre duas coisas. Não cabe dentro de uma delas.

A mesma ordem no Nystrom

```
void World::gameLoop() {  
    while (true) {  
        // Handle user input...  
        // Update each entity.  
        for (int i = 0; i < numEntities_; i++) {  
            entities_[i]->update();  
        }  
        // Physics and rendering...  
    }  
}
```

Física vem depois do for. A nossa colisão é aquele comentário.

O placar desenha, mas não atualiza

Se ele herdasse de Entidade, teria um `update()` vazio só para cumprir o contrato.

Então ele fica de fora da lista: é uma classe, mas não é uma entidade.

Nem tudo que virou classe virou entidade.

Quantas linhas do laço mudam
para o jogo ter duas bolas?

Nenhuma

```
undo.entidades.append(Bola())
```

O laço não sabe quantas entidades existem. E não precisa saber.

Refatoração não conserta lógica

```
if self.y + self.ALTURA / 2 < self.bola.y:  
    self.y += self.VELOCIDADE * dt
```

```
if self.y + self.ALTURA / 2 > self.bola.y:  
    self.y -= self.VELOCIDADE * dt
```

Quando a bola chega perto, o oponente desce e sobe no mesmo quadro. Trepidava antes, trepida agora.

E uma raquete que também quica na parede: de quem ela herda?

Onde a herança começa a ranger

A hierarquia decide tudo na hora em que a classe é escrita.

E a World cresce a cada coisa que não pertence a ninguém:
física hoje, som e pontuação depois.

Física, gráficos, entrada. Guardem essas três palavras.

A PRÓXIMA

O Pong em uma engine

Godot, Unity e Unreal já vêm com entidades prontas.

Vamos olhar o que cada uma chama de entidade e de mundo.

Para casa

refatorar o seu Pong para classes — âmbar: pode usar IA, precisa explicar cada classe

reler o capítulo Update Method olhando para a classe World